

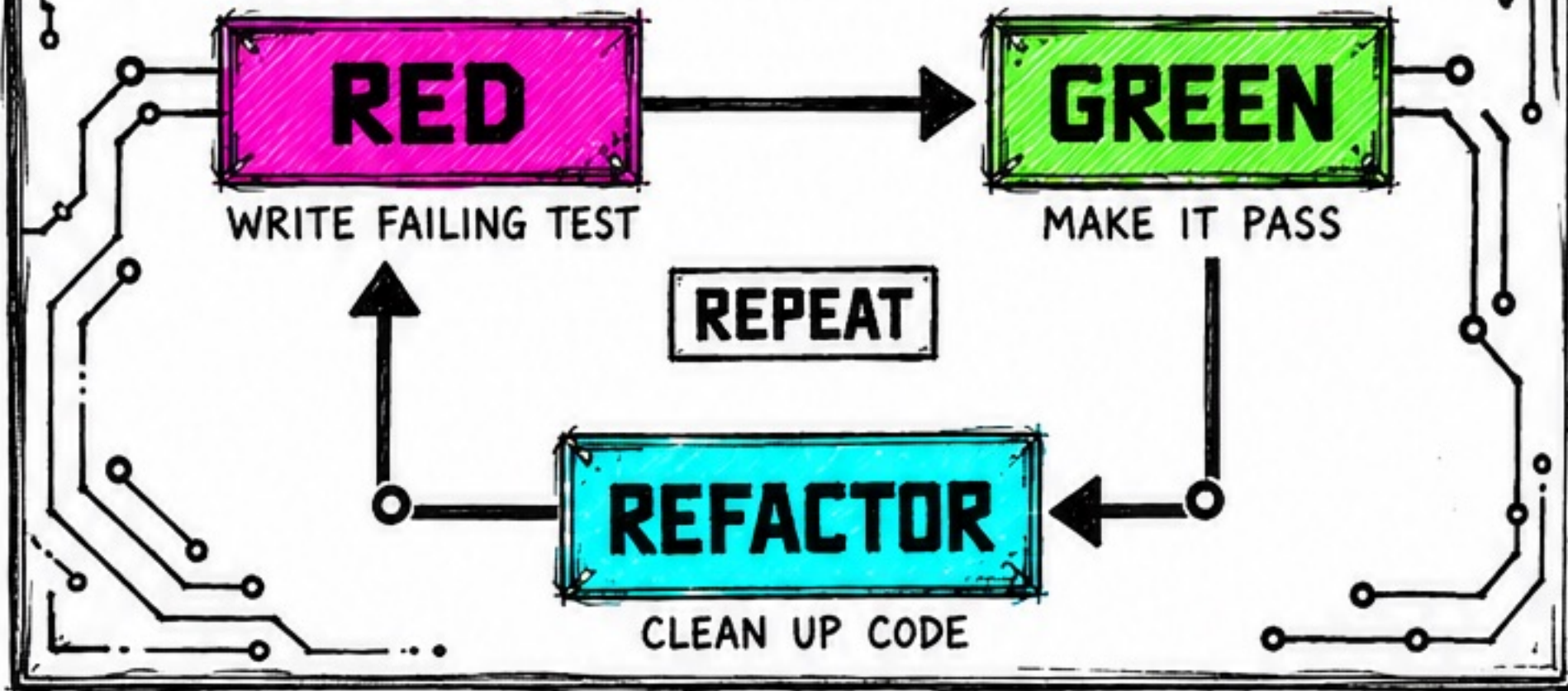
RED GREEN REFACTOR

REPLACE HOPE WITH REPEATABLE CHECKS

WHY TEST

- > HOPE IS NOT A STRATEGY
- > ASSERT: CODE BEHAVES AS EXPECTED
- > REGRESSION = WORKED, NOW BROKEN
- > NO TESTS → FEAR, TECH DEBT

TDD CYCLE



TEST PYRAMID



PYTEST RULES

- > PLAIN assert, LESS BOILERPLATE
- > FILES: test_*.py OR *_test.py
- > FUNCTIONS: test_* CLASSES: Test*
- > tests/ SEPARATE + __init__.py

READ THE FAILURE

- > . PASS **F** FAIL **E** ERROR s SKIP
- > AssertionError: EXPECTED VS ACTUAL
- > TRACEBACK = FILE + LINE
- > A FAILURE IS A BUG REPORT

COMMANDS

```
pip install pytest
pytest --version
pytest
> _
```

test_ PREFIX
OR PYTEST
FINDS NOTHING

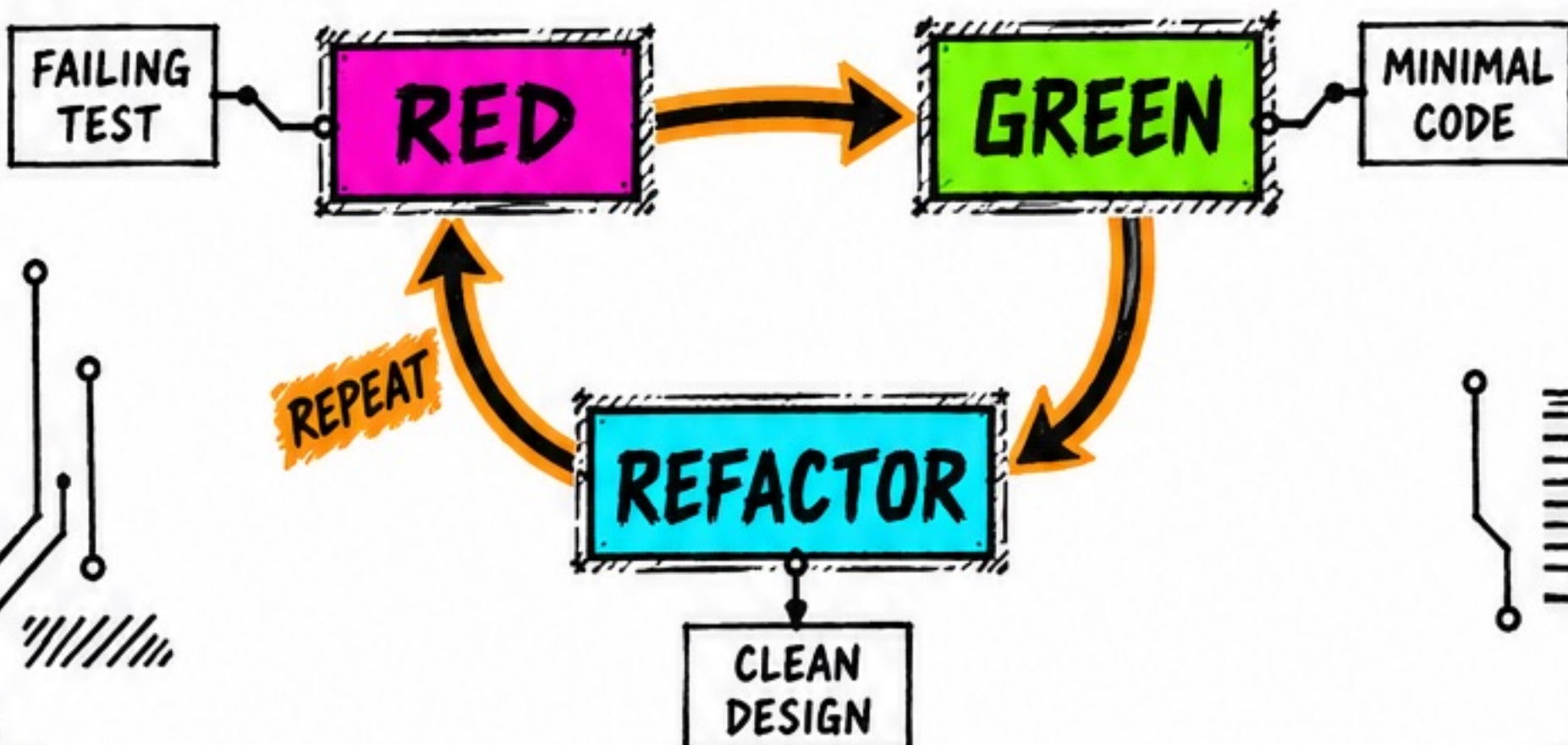
RED GREEN REFACTOR

TEST FIRST. PASS FAST. CLEAN UP.

WHY TDD

- > Tests written late mean late debugging
- > Code first breeds tight coupling
- > Fast feedback catches bugs immediately
- > Kent Beck, Extreme Programming, 1990s

THE LOOP



RED

- > Write a failing test first
- > Proves the feature is missing
- > Validates the test itself
- > Specific name, one concept

GREEN

- Minimum code to pass
- Hardcoding is allowed
- Not cheating, not final
- Avoid over-engineering now

REFACTOR

- ⊕ Change how, never what
- ⊕ **IS:** remove duplication, rename, simplify
- ⊕ **NOT:** features, behavior, dependencies
- ⊕ Passing tests are the safety net

COMMANDS

```
test_add_task_with_empty_title() # good
test_tasks() # too broad
```

NOTE TO SELF

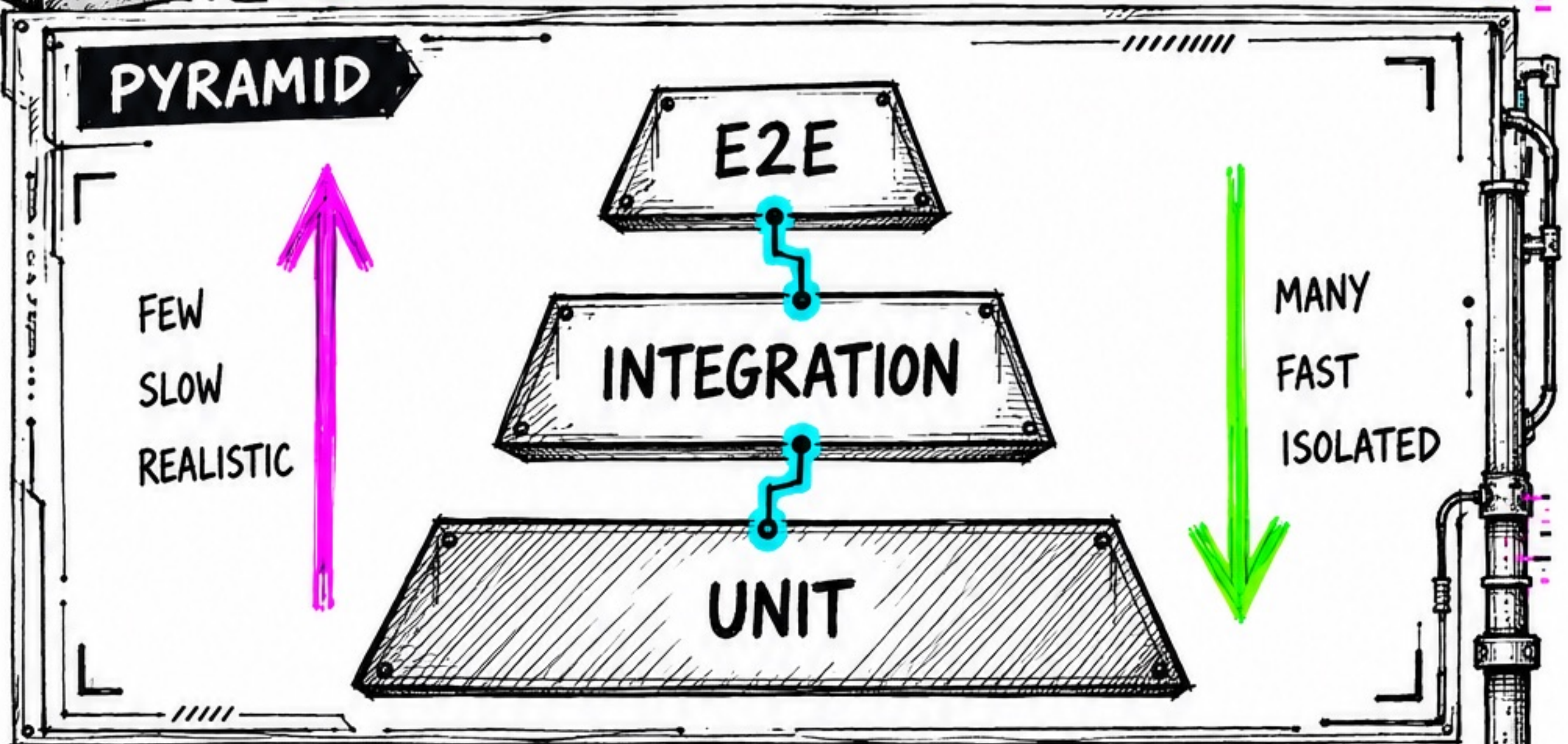
A test that never failed proves nothing





THE TESTING PYRAMID

RIGHT TEST, RIGHT LAYER, RIGHT TIME



UNIT TESTS

- > isolated: no db, network, files
- > milliseconds, thousands of them
- > failure points to exact unit
- > happy path, edge cases, business rules

INTEGRATION TESTS

- > test the seams between components
- > real db, api, file system
- > catch data contract bugs
- > fewer than unit, more than e2e

E2E TESTS

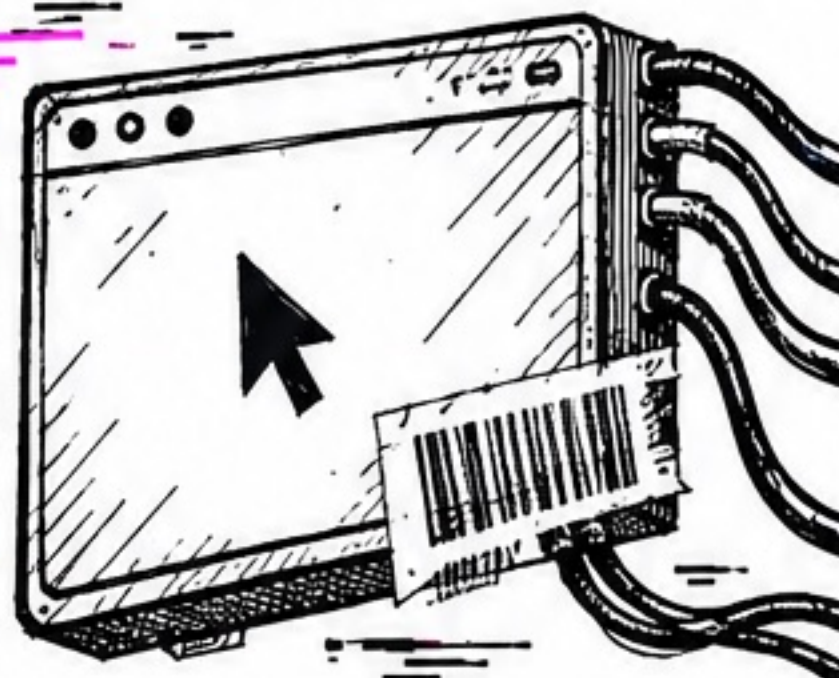
- > whole system, user's perspective
- > browser automation: selenium, playwright
- > slowest, most fragile, costly
- > only critical user journeys

STRATEGY

- no test type is better
- together: a safety net
- which test gives most value now?
- quality over quantity

NOTE TO SELF

Many fast tests below, few slow above.
Start with one happy path.



CI/CD PIPELINE

EVERY PUSH BUILDS, TESTS, SHIPS

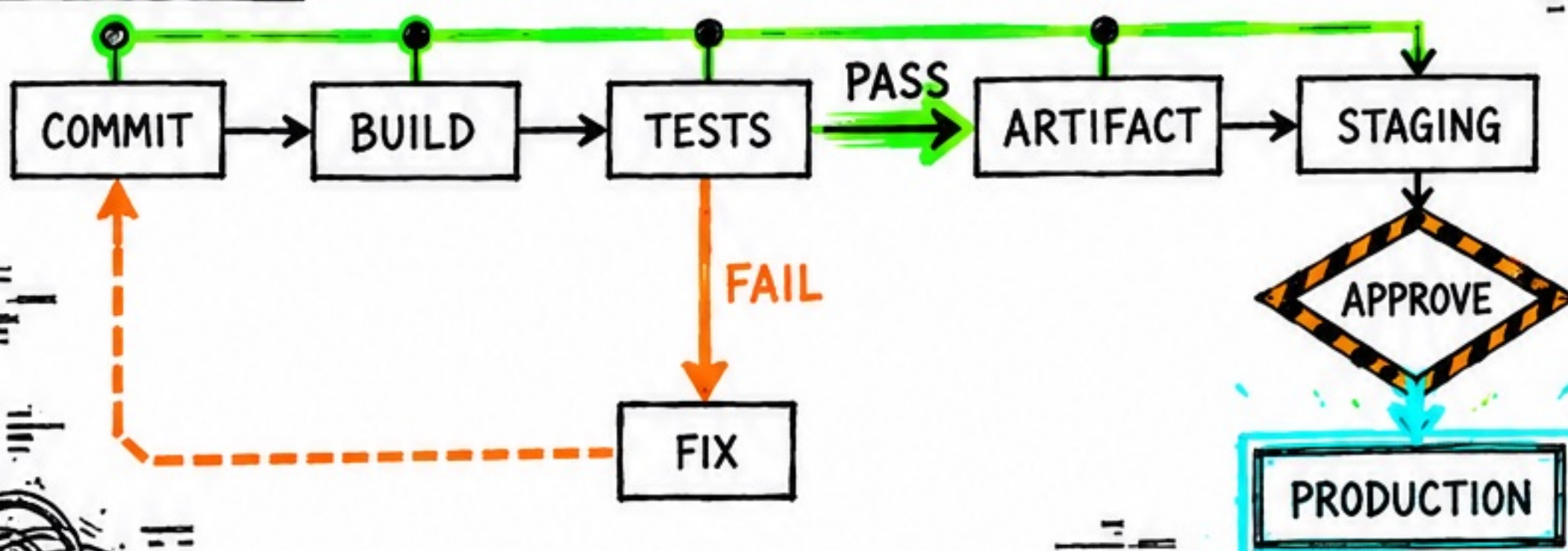
CONTINUOUS INTEGRATION

- > Merge to shared repo daily
- > Every push builds and tests
- > Feedback in minutes, fix now
- > No more integration hell

DELIVERY VS DEPLOYMENT

- **Delivery**: artifact to staging
- Human approves the release
- **Deployment**: auto to production
- Fail? Fix and commit again

PIPELINE



WORKFLOW FILE

- > Lives in `.github/workflows`
- > Trigger: push or pull request
- > Steps: checkout, install, test
- > Green check = passing

TOOLS

- ⊕ **GitHub Actions**: in repo
- ⊕ **GitLab CI/CD**: in repo
- ⊕ **CircleCI**: simple
- ⊕ **Jenkins**: powerful, custom

COMMANDS

```
.github/workflows/ci.yml  
npm install
```

Delivery stops at staging.
Deployment goes live.

TDD MEETS AI

YOU OWN THE SPEC, AI FILLS CODE

TESTS AS SPEC

- AI lacks your business context
- Tests encode exact requirements
- Living spec, whatever wrote the code
- Write test first, AI passes it

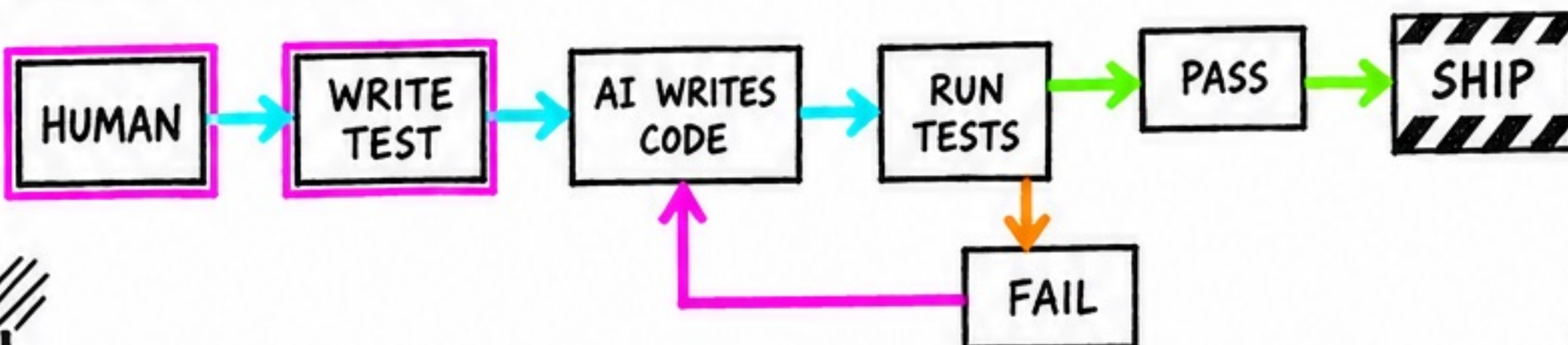
AI-WRITTEN TESTS

- Match implementation, not requirements
- Miss edge cases and business rules
- Shallow coverage, false safety
- Code first, test later: **not TDD**

THE SWEET SPOT

- You own critical paths, business logic
- AI handles boilerplate and exploration
- Be intentional, **keep ownership**

WORKFLOW



AI IN CI/CD

TOOL	EFFECT
PREDICTIVE SELECTION	run tests likely to fail
FLAKY DETECTION	quarantine intermittent tests
FAILURE CLUSTERING	group by stack trace
SMART DIAGNOSTICS	logs, severity, PR update

NOTE TO SELF

AUTO TESTS MIRROR CODE,
NOT REQUIREMENTS